Validating Optimizing SMT Solvers via Cross-Theory Approximation

MAOLIN SUN, Nanjing University, China
FUQI JIA, Institute of Software at Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China
YIBIAO YANG*, Nanjing University, China
YUMING ZHOU, Nanjing University, China

Optimization Modulo Theories (OMT) extends Satisfiability Modulo Theories (SMT) by integrating logical reasoning with numerical optimization to address constrained optimization problems across diverse theories. Optimizing SMT solvers (also known as OMT solvers), designed to handle such problems, serve as foundational components in numerous applications within programming languages research and practice. However, despite their widespread adoption, OMT solvers are susceptible to subtle yet critical bugs that can silently mislead downstream applications by providing incorrect optimal solutions, potentially leading to severe consequences. Validating these solvers poses a fundamental challenge, as obtaining precise ground truth for optimal solutions is inherently difficult, particularly under complex or theory-specific objective functions. Moreover, existing SMT solver testing techniques are inadequate, as they fail to capture the intricate interplay between satisfiability checking and optimization reasoning in OMT. To overcome these challenges, we propose ***cross-theory approximation***, a novel validation methodology that leverages the relationships between solution spaces of different logical theories. The key insight is that an optimal solution produced in one theory should maintain expected relationships when interpreted in another comparable theory's solution space. By defining these cross-theory consistency properties and comparing optimal solutions obtained through theory-specific transformations, we can detect discrepancies that indicate potential solver bugs. For instance, an integer-optimal solution should map cleanly into the broader real-arithmetic domain; deviations from this expected relationship signal incorrect optimization behavior. We implement this methodology in Iris, a practical framework for validating OMT solvers. When testing advanced OMT solvers, including Z3 and OptiMathSAT, Iris uncovered 24 previously unknown bugs, 20 of which were subsequently resolved by developers. Notably, most of our reported bugs are correctness issues, emphasizing the effectiveness of our approach in enhancing OMT solver reliability.

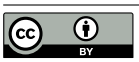
CCS Concepts: • **Software and its engineering → Formal methods**; **Software testing and debugging**.

Additional Key Words and Phrases: OMT solver, validation, approximation

ACM Reference Format:
Maolin Sun, Fuqi Jia, Yibiao Yang, and Yuming Zhou. 2026. Validating Optimizing SMT Solvers via Cross-Theory Approximation. *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 405 (October 2026), 27 pages. https://doi.org/10.1145/3839537

*Corresponding author.

Authors' Contact Information: Maolin Sun, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, merlin@smail.nju.edu.cn; Fuqi Jia, Key Laboratory of System Software, Institute of Software, Chinese Academy of Sciences, Beijing, China, jiafq@ios.ac.cn; Yibiao Yang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, yangyibiao@nju.edu.cn; Yuming Zhou, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, zhouyuming@nju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2475-1421/2026/10-ART405
https://doi.org/10.1145/3839537

1 Introduction

Satisfiability Modulo Theories (SMT) is a computational problem that focuses on determining whether first-order logical formulas are satisfiable within the context of one or more background theories [7]. While SMT solvers are effective at checking the existence of solutions, many real-world applications require more. For example, they need solutions that are not just feasible, but optimal, according to specific objective functions. This need has driven the emergence of Optimization Modulo Theories (OMT), an extension of SMT that integrates decision procedures with numerical optimization techniques [50]. OMT problems combine constraints from a given theory with the goal of optimizing a specified term, and optimizing SMT solvers (*a.k.a.* OMT solvers) have been developed to address these challenges [10]. For instance, given the constraint $x \geq 0$, an SMT solver can confirm a solution exists (e.g., $x = 10$), while an OMT solver goes further by finding the optimal value of the specified term, such as x's minimum value $x = 0$. The ability to compute optimal solutions has made OMT solvers valuable across diverse domains in programming languages and systems research. For example, they have been successfully applied to tasks such as program synthesis [20, 22, 24, 33], program analysis [56], fuzzing [23], side-channel analysis [19], formal verification [45], quantum compiler optimization [38, 39], and differential privacy [28, 58]. Recent studies have further applied OMT-based techniques to constrained subgroup discovery [4] and robot preference learning [37], demonstrating the continued relevance of OMT solvers in emerging application domains.

However, like any sophisticated software, OMT solvers are susceptible to bugs that can lead to incorrect results, undermining the applications that depend on them. For instance, a bug in an OMT solver used for vulnerability analysis could cause critical security flaws to go undetected [23]. In some cases, bugs in OMT or SMT solvers themselves may even be exploitable as security vulnerabilities [1–3]. Given the broad use of OMT solvers, ensuring their correctness and reliability is critical. In practice, validating OMT solvers presents a unique and formidable challenge that extends beyond the scope of traditional SMT solver testing. Specifically, an SMT solver primarily needs to determine whether a formula is satisfiable, while an OMT solver must also find an optimal solution for a specified objective function [50]. Therefore, validation must check not only whether the returned result is feasible, but also whether the reported optimum is correct. This additional requirement substantially complicates the design of effective test oracles.

Existing SMT validation techniques, such as differential testing, often validate solver correctness by comparing results against a complete reference implementation [47, 52]. Nevertheless, obtaining such a reference solver for OMT remains difficult. For example, despite its widespread adoption, Z3 does not officially support non-linear optimization problems [10], and the optimization extension of SMT-LIB is still under standardization [51]. As a result, unsupported or partially supported problem fragments may produce `unknown` results, timeouts, or solver-specific behaviors that are difficult to compare reliably. Moreover, extending reference-independent SMT testing techniques to OMT is non-trivial. Many such techniques rely on mutation strategies that may generate problem instances outside the supported capabilities of existing OMT solvers, limiting their practical effectiveness [48]. Another promising direction is *solution verification*, which checks whether a reported solution satisfies the constraints and whether the claimed objective value is indeed optimal. However, solution verification alone cannot generate new inputs to exercise solver behavior and is difficult to apply when the correct result is not an attained optimum, such as cases involving unattained infima, suprema, or unbounded objectives.

Our empirical study of Z3 optimization issues in Section 2 further demonstrates the need for OMT-specific validation techniques. By analyzing 240 optimization-related issue reports from Z3's

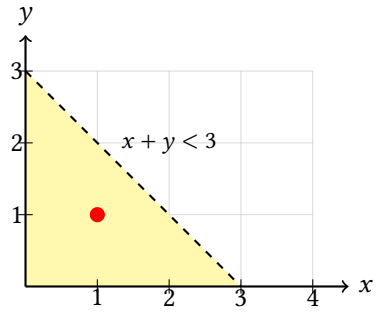


Fig. 1. Solution spaces for $x > 0 \wedge y > 0 \wedge x + y < 3$ under real and integer arithmetic. The yellow region represents the real solution space, and the red point denotes the integer solution.

public issue tracker, we identified several recurring categories of failures, including incorrect optimization results, soundness bugs, crashes, and performance issues. Among these categories, incorrect optimization results and soundness bugs are particularly concerning because they can silently compromise the correctness of downstream applications. Although incorrect optimization results represent a relatively small portion of reported issues, we conjecture that this reflects the lack of effective techniques for exposing such failures rather than their inherent rarity. These findings suggest that validating OMT solvers requires techniques that can detect violations of optimization semantics, rather than merely checking satisfiability outcomes or solver robustness. Motivated by this observation, we investigate a new validation strategy based on the fact that the same OMT problem can often be expressed under multiple theories while preserving its intended meaning. For example, consider the formula $x > 0 \wedge y > 0 \wedge x + y < 3$. Depending on the application context, this formula can be interpreted over either real or integer arithmetic.

Our Approach. In this work, we propose a novel validation technique for OMT solvers based on ***cross-theory approximation***. The key idea is to identify solver bugs by comparing the optimal solutions of logically equivalent or near-equivalent formulas expressed over different theories. Specifically, given a first-order formula, we incorporate objective functions that instruct the solver to reason about optimal solutions. These functions enable optimization in OMT solvers by requesting the maximum or minimum value of a specified term. To ensure diversity and semantic relevance, these objective terms are generated using theory-aware grammars that span multiple common SMT theories. Next, we apply theory-specific approximations to transform the original formula while preserving its semantics as closely as possible. This transformation mutates the formula's theory sorts, generating new formulas that maintain near-equivalent meaning across different theories. For example, we can approximate formulas between discrete domains, such as integers, and continuous domains (e.g., reals). Since real arithmetic allows a broader solution space than integer arithmetic, this transformation establishes predictable relationships between optimal values on corresponding objective functions. In this context, the optimal value in the integer domain cannot be "*better*" (e.g., lower for minimization, higher for maximization) than that in the real domain. Ultimately, we input both the original and transformed formulas into OMT solvers and compare their optimal values. Any deviations that violate the expected relationships between different theories indicate potential solver bugs.

Implementation. Designing and implementing effective cross-theory approximations is non-trivial. While theories like integers and reals share many syntactic constructs, their semantic interpretations can differ in subtle but important ways. For example, operators like division behave differently in each theory: integer division truncates results, while real division produces exact quotients. Our approach addresses these challenges by carefully selecting and approximating formulas to preserve logical structure and avoid introducing undefined or lossy transformations. Concretely, we define transformation rules that maintain meaningful correspondence between the original and approximated formulas, enabling sound reasoning about the expected relationships between their optimal values. Ultimately, we realize our technique as a practical tool, Iris, and apply it to four advanced OMT solvers, i.e., Z3 [10], OptiMathSAT [35, 44], Symba [31], and OCAC [25]. Our evaluation demonstrates that Iris is effective in uncovering bugs in OMT solvers, including correctness issues. To date, Iris has identified 24 previously unknown bugs in the solvers; of these, 21 have been confirmed and 20 have been fixed by the developers. Notably, multiple bugs have been latent in the solvers for years, underscoring the importance of our approach in enhancing OMT solver reliability. Moreover, Iris has demonstrated its capability to exercise the optimization functionalities of the solvers, achieving a substantial improvement in code coverage.

Contributions. Our main contributions are as follows:

- ***Novelty:*** We shed light on the critical aspect of OMT solver validation and propose a novel approach based on cross-theory approximation to identify bugs in OMT solvers. To the best of our knowledge, this is the first dedicated approach for OMT solver validation.
- ***Significance:*** Our work tackles the challenging problem of effectively validating OMT solvers, which are essential for ensuring the correctness and robustness of the solvers. Notably, our approach primarily focuses on correctness issues, which are critical for OMT solvers and their applications.
- ***Usefulness:*** We have implemented Iris's prototype and used it to uncover 24 issues in four leading OMT solvers, such as Z3 and OptiMathSAT. Of these, 21 issues have been confirmed or resolved by the developers. We have also made Iris publicly available to facilitate further research in this area.

Paper Organization. The rest of this paper is structured as follows. Section 2 introduces the background and illustrates our approach using a concrete example. Section 3 formalizes our approach. Next, we present our evaluation in Section 4, and Section 5 further discusses the results. Finally, we survey related work in Section 6 and conclude in Section 7.

2 Preliminary and Illustrative Example

In this section, we first provide a brief overview of OMT solvers, the SMT-LIB language with OMT extensions, and the theories supported by OMT solvers. Then, we delve into the process of cross-theory approximation using an illustrative example.

2.1 Preliminaries

Satisfiability and Optimization Modulo Theories Solvers. SMT solvers automate reasoning about logical formulas that are augmented with background theories, determining whether a set of constraints is satisfiable (*sat*) or unsatisfiable (*unsat*). Since there is extensive literature providing

formal descriptions of SMT problems [7], we focus here on the definition most relevant to this work.

Definition 2.1 (Theories in SMT). *A theory is a pair $T = (\Sigma, A)$, where Σ is a signature consisting of sort symbols $\{S_1, S_2, ...\}$ (types) and function symbols $\{F_1, F_2, ...\}$, and A is a class of Σ-interpretations.*

Informally, a theory defines specific types and functions, restricting how symbols in Σ can be interpreted.

Though SMT solvers are widely used, many real-world applications require more than just a binary *sat*/*unsat* result. Practical problems often demand solutions that optimize specific objectives, such as minimizing costs and maximizing rewards within the constraints. Thus, OMT is devised to extend SMT by adding the ability to find not just any solution but one that optimizes a specified goal. Here, we adopt the definition of Generalized Optimization Modulo Theories (GOMT) from prior work [50].

Definition 2.2 (Generalized Optimization Modulo Theories (GOMT)). *A GOMT problem is a tuple $GO := \langle t, \prec, \varphi \rangle$, where:*

- *t is a Σ-term of some sort σ, representing the objective term to optimize;*
- *$\prec$ is a strict partial order definable in T, whose defining formula has two free variables, each of sort σ;*
- *φ is a Σ-formula, representing the constraints on the values that t can take.*

For any GOMT problem GO and interpretations I and I', we define the following:

- *I is **GO-consistent** if it satisfies φ (i.e., $I \vDash \varphi$), indicating that I is a solution (a.k.a. model) to φ;*
- *I **GO-dominates** I', denoted by $I <_{GO} I'$, if I and I' are GO-consistent and $t_I \prec t_{I'}$;*
- *I is a **GO-solution** if I is GO-consistent and no interpretation I' GO-dominates I.*

Informally, the term t represents the objective function to optimize, $\prec$ defines what "*better*" means when comparing solutions, and φ specifies the constraints that the objective function must satisfy. A solution I is considered optimal if it satisfies the constraints and no other interpretation I' dominates it (i.e., *better* than it) according to the optimization criterion. For simplicity, we refer to a GO-consistent interpretation as a solution and a GO-solution as an optimal solution for a GOMT problem.

SMT-LIB Language with OMT Extensions. The widely adopted input language for OMT solvers is based on SMT-LIB [6], which provides a comprehensive standard for describing problems across various theories [51]. Key commands in SMT-LIB include `declare-fun`, `assert`, and `check-sat`. To illustrate, consider the example in Figure 2. The command `(declare-fun a () Int)` defines a 0-arity function of `Int` sort (e.g., an integer variable a), and `(declare-const a Int)` could serve the same purpose here. The `assert` command then adds constraints, such as requiring a to satisfy $1 < (0-a)-a$. The `check-sat` command instructs the solver to check whether all constraints can be met, returning `sat` if a solution exists or `unsat` otherwise. To support optimization, certain commands are added to the SMT-LIB language for OMT solvers. For instance, `minimize` and `maximize` allow users to define objective functions to optimize, such as finding the smallest or largest value of a variable [10], and `get-objectives` retrieves the optimal values found by the solver, which are then reported as the final result.

```
(set-logic QF_LIA)
(declare-fun a () Int)
(assert (< 1 (- (- 0 a) a)))
(check-sat)
```

Fig. 2. Original formula φ.

Theories Supported by OMT Solvers. The rapid development of OMT solvers has led to varying degrees of optimization support across different logical theories. Early OMT solvers addressed linear arithmetic and bit-vector theories, which form foundational components in many verification applications [10]. A significant advancement occurred with OptiMathSAT [35, 44], which extended the OMT solvers to non-linear arithmetic. Recent work by Tsiskaridze et al. [50] has introduced a more generalized OMT formulation, which is promising for handling a broader range of optimization problems. For theories supported by modern OMT solvers, their inter-theoretic relationships can be characterized through the nature of their domains. For instance, the domain of bit-vectors is finite, while the domain of integers is infinite. For infinite domains, further distinctions can be made between discrete and continuous domains (e.g., integers versus reals). We now define the *nature of the domain* of a theory's sorts to facilitate classifying their properties and relationships.

Definition 2.3 (Finite-Domain Theory). *A theory $T = (\Sigma, A)$ is a finite-domain theory if, for every sort $S \in \Sigma$, the cardinality $|S|$ is finite.*

Definition 2.4 (Infinite-Domain Theory). *A theory $T = (\Sigma, A)$ is an infinite-domain theory if there exists some sort $S \in \Sigma$ such that the cardinality $|S|$ is infinite.*

Additionally, among infinite-domain theories, further refinement can be made. For instance, discrete infinite-domain theories such as integer arithmetic have a successor structure, while continuous infinite-domain theories (e.g., real arithmetic) have uncountably infinite domains with elements densely packed between any two distinct elements. Based on the nature of their domains, we can utilize the relationships between theories to validate OMT solvers through cross-theory approximation, as we will illustrate in the following example.

Bug Categories in OMT Solvers. To better understand the types of bugs that occur in OMT solvers, we conducted an empirical study using the public issue tracker of Z3 [36], one of the most widely adopted solvers in the community. Its open-source nature provides access to detailed bug reports and developer discussions. We focused specifically on issues related to optimization by searching for keywords such as *maximize* and *minimize*. This process yielded 240 issue reports that explicitly referenced optimization functionalities, spanning the period from 2015 to 2025. Each issue was manually classified following open coding practices [43]. Specifically, two researchers with relevant experience independently reviewed the symptoms and assigned categories. Disagreements were resolved through discussion with a third researcher, ensuring reliable classifications.

Finally, we categorized these issues into five major groups based on their observed symptoms:

- **Optimization Correctness Bugs (8.75%):** The solver returns sub-optimal solutions that violate the expected optimality guarantee. These bugs are particularly critical but difficult to detect, affecting downstream applications silently.
- **Soundness Bugs (17.08%):** These involve incorrect logical behavior, such as reporting `unsat` instead of `sat`, or producing invalid models when optimization commands are used. Like the first category, these issues are severe and may silently affect downstream tasks.

```
(declare-fun a () Int)
(assert (< 1 (- (- 0 a) a)))
(minimize (- (- 0 a) a))
(check-sat)
(get-objectives)
```

(a) Formula with objective functions φ_o.

```
(declare-fun a () Real)
(assert (< 1.0 (- (- 0.0 a) a)))
(minimize (- (- 0.0 a) a))
(check-sat)
(get-objectives)
```

(b) Theory-specific transformed formula φ_t.

```
$ optimathsat int.smt2
sat

(objectives
(- (- 0 a) a) → 2
)
```

(c) Optimal value for formula φ_o shown in Figure 3a.

```
$ optimathsat real.smt2
sat

(objectives
(- (- 0.0 a) a) → 3
)
```

(d) Optimal value for formula φ_t shown in Figure 3b.

Fig. 3. Cross-theory approximation validation illustrated. (a) Seed formula φ; (b) Extended formula φ_o with an optimization objective; (c) Transformed formula φ_t in the theory of real numbers. The OMT solver reports conflicting minima for the objective function `(- (- 0 a) a)`: (d) Minimum value 2 under integer arithmetic; (e) Minimum value 3 under real arithmetic. Results are highlighted with colored backgrounds: **blue** indicates theoretically correct solutions, while **red** highlights incorrect solutions. The observed inversion ($min_{Real} > min_{Int}$) violates the expected dominance relationship between continuous and discrete optimization for the objective function `(- (- 0 a) a)`.

- **Crashes (37.50%):** These bugs result in solver crashes or segmentation faults. While generally less critical than correctness issues, they can still disrupt user workflows. In many cases, however, such crashes are triggered by invalid solver configurations, which are unlikely to occur in practical use. For instance, one reported issue #3660 is caused by an unsupported combination of three solver options and could likely have been identified through fuzz testing.
- **Performance Issues (8.33%):** These bugs cause excessive memory use or unacceptably long solving times. While not always severe, they can degrade user experience.
- **Other Issues (28.34%):** This residual category includes API issues, feature requests, and miscellaneous behavior not captured above.

Notably, while bugs involving incorrect optimization results appear relatively infrequently, we hypothesize that this is due to the lack of dedicated validation techniques for OMT solvers, rather than an indication of their rarity. Given their potential to compromise the correctness of downstream applications, we focus our work primarily on detecting such bugs. We also consider soundness bugs and crashes where appropriate.

2.2 Illustrative Example

To illustrate the process of cross-theory approximation validation, we present a concrete example, as depicted in Figure 3, based on a real-world bug discovered in OptiMathSAT using our approach. The procedure is organized into three steps:

Step 1: Objective Function Arrangement. To activate the solver's optimization capabilities, it is necessary to first include an objective function for optimization. For instance, consider the formula φ in Figure 2, which enforces the constraint `(< 1 (- (- 0 a) a))`, equivalent to the

inequality $1 < -2a$ in mathematical terms. Without an objective function, the solver will merely determine the satisfiability of the formula, which does not exercise its optimization capabilities. To engage its optimization capabilities, we introduce the objective function to minimize the term `(- (- 0 a) a)`, i.e., $-2 * a$, resulting in the modified formula φ_o (Figure 3a). This adjustment enables the solver to search for optimal solutions in addition to checking satisfiability.

Step 2 Theory-specific Transformation: To facilitate cross-theory approximation validation, we transform the formula φ_o into a version with nearly equivalent semantics in a different theory. For instance, in this example, we transform the formula φ_o from integer arithmetic to real arithmetic, yielding the formula φ_t shown in Figure 3b. Typically, this transformation does not guarantee the strict equivalence of the formulas' semantics, thereby potentially affecting the satisfiability and optimality results. However, it should also maintain certain properties due to the nature of the domains in theories, making the following examination of the solver's behavior across different theories possible.

Step 3 Approximation Property Validation: The ultimate step is to validate the solver's behavior by comparing the solutions of φ_t (real arithmetic) and φ_o (integer arithmetic). In this example, both formulas remain satisfiable since $1 < -2a$ has solutions in both theories. However, the solution sets for variable a diverge. Concretely, for φ_o, the solution set is $\{a \in \mathbb{Z} \mid a \leq -1\}$. For formula φ_t, the solution set is $\{a \in \mathbb{R} \mid a < -0.5\}$, suggesting a border solution space. Consequently, in theory, minimizing $-2a$ cannot produce a lower bound in integer arithmetic. However, OptiMathSAT produces contradictory results. As shown in Figures 3c and 3d, it reports minimum values of 2 for φ_o and 3 for φ_t. This indicates that the objective function achieves a *better* optimal value in the integer case than in the real case, which contradicts theoretical expectations and thus exposes a correctness bug in OptiMathSAT. This issue was reported by Iris and subsequently resolved by the developers.

3 Approach

In this section, we first provide a comprehensive overview of our proposed approach, followed by a detailed description.

3.1 Overview

This study introduces Iris, a novel technique for validating OMT solvers through cross-theory approximation analysis. As illustrated in Figure 4, Iris's validation pipeline operates through three key phases. It first leverages existing SMT formulas as foundational input. For each seed formula, Iris extends the problem by adding objective functions using predefined grammars to activate the optimization capabilities of solvers. Subsequently, Iris applies theory-specific formula transformations, mutating the sorts of original formulas and generating semantically comparable formulas. Finally, by executing OMT solvers on both original and transformed versions, Iris inspects behavioral inconsistencies that deviate from expected solution relationships, exposing solver bugs. Algorithm 1 provides detailed pseudocode for Iris's testing process, which we elaborate on below.

3.2 Objective Function Arrangement

Given a seed formula, Iris augments the instance by incorporating objective functions that activate the solver's optimization capabilities (Line 4 in Algorithm 1). For each seed, Iris generates at most five objective commands, each chosen as either a `maximize` or `minimize` directive. To ensure that the generated objective functions are both valid and diverse, Iris employs a grammar-based generation strategy. Figure 5 illustrates the grammars used for the `Ints` and `Reals` theories, which cover both linear and non-linear arithmetic, as well as for the `BitVectors` theory. These grammars are adapted from the design of ET [52]. To illustrate the process, consider the example shown in

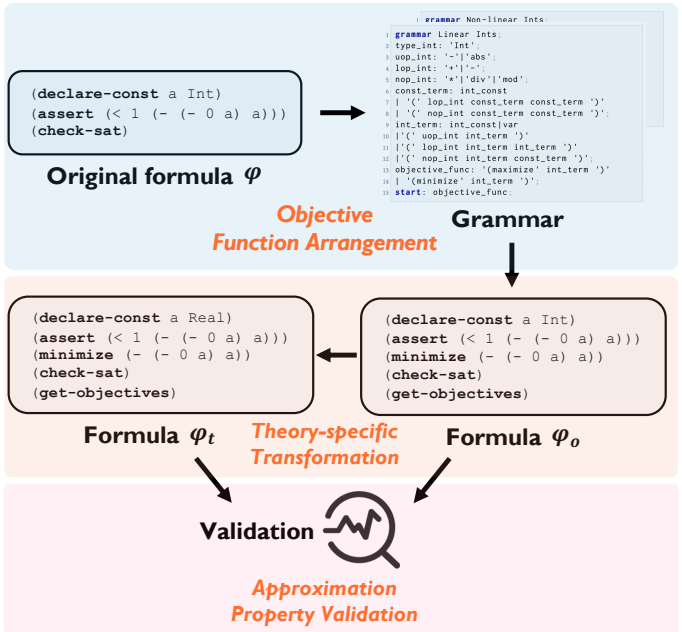


Fig. 4. Overview of Iris's validation process.

Figure 2, where Iris generates the objective function `(minimize (- (- 0 a) a))`. Iris proceeds by selecting a grammar based on the logic of the seed formula, which in this case is linear arithmetic over integers, and starts with the `start` rule shown in Grammar 4. It then randomly selects a target command, either `(maximize int_term)` or `(minimize int_term)`. Assume `(minimize int_term)` is chosen. Here, `int_term` represents a non-terminal symbol that can be expanded into various terms, e.g., `` `(' lop_int int_term int_term `)' ``. Subsequently, Iris instantiates the non-terminal symbol `lop_int` by selecting a random binary operator from the candidates, such as `-`. This process continues iteratively, with non-terminal symbols being progressively instantiated until the full objective function is formed. During generation, `int_const` and `real_const` are instantiated with randomly generated constants, while `var` is restricted to variables already appearing in the seed formula. To avoid overly complex objectives, Iris limits the maximum generation depth to 10 and discards candidates that reduce to a single variable or literal when a more complex objective can be generated. Once all non-terminal symbols are instantiated, Iris generates a fully instantiated objective function and appends it to the seed formula, forming the basis for the subsequent validation phase. The grammars in Figure 5 describe the general generation strategy; before emitting SMT-LIB formulas, Iris performs additional theory-specific checks to ensure that generated expressions are well-formed. This step is particularly important for the `BitVectors` theory, where Iris tracks the bit-width of each expression and ensures that operands of binary bit-vector operators, such as `bvadd` and `bvmul`, have matching widths. These checks prevent invalid expressions while maintaining the diversity of generated objective functions.

3.3 Theory-Specific Transformations

To generate effective test formulas with predictable outcomes for validation, Iris applies specialized transformations that mutate the sorts (i.e., data types) of variables and terms in seed formulas.

```
grammar Non-linear Ints;
uop_int: `-'|`abs';
bop_int: `-'|`+'|`*'|`div'|`mod';
int_term: var|`(' uop_int int_term `)'
|`(' bop_int int_term int_term `)';
objective_func: `(maximize' int_term `)'
| `(minimize' int_term `)';
start: objective_func;
```

Grammar 1. Non-linear Ints

```
grammar Non-linear Reals;
uop_real: `-';
bop_real: `-'|`+'|`*'|`/';
real_term: var|`(' uop_real real_term `)'
| `(' bop_real real_term real_term `)';
objective_func: `(maximize' real_term `)'
| `(minimize' real_term `)';
start: objective_func;
```

Grammar 2. Non-linear Reals

```
grammar Bitvectors;
uop_bv: `bvnot'|`bvneg';
bop_bv: `bvand'|`bvor'|`bvadd'|`bvmul'
|`bvudiv'|`bvurem'|`bvshl'|`bvlshr';
bv_term: bv_const|`(' uop_bv bv_term `)'
|var| `(' bop_bv bv_term bv_term `)';
objective_func: `(maximize' bv_term `)'
| `(minimize' bv_term `)';
start: objective_func;
```

Grammar 3. Bitvectors

```
grammar Linear Ints;
uop_int: `-';
lop_int: `+'|`-';
nop_int: `*';
const_term: int_const
| `(' lop_int const_term const_term `)'
| `(' nop_int const_term const_term `)';
int_term: var|`(' uop_int int_term `)'
|'(' lop_int int_term int_term `)'
|'(' nop_int int_term const_term `)';
objective_func: `(maximize' int_term `)'
| `(minimize' int_term `)';
start: objective_func;
```

Grammar 4. Linear Ints.

```
grammar Linear Reals;
uop_real: `-';
lop_real: `+'|`-';
nop_real: `*`;
const_term: real_const
| `(' lop_real const_term const_term `)'
| `(' nop_real const_term const_term `)';
real_term: var|`(' uop_real real_term `)'
|`(' lop_real real_term real_term `)'
|`(' nop_real real_term const_term `)';
objective_func: `(maximize' real_term `)'
| `(minimize' real_term `)';
start: objective_func;
```

Grammar 5. Linear Reals.

Fig. 5. Objective function grammars for different theories.

This process yields new formulas with nearly equivalent semantics (Line 5). In the following, we first formally define the Sort-Mutated GOMT (SM-GOMT) problem and establish its key properties. We then describe the specific transformation strategies implemented in Iris for converting between distinct logical theories.

Sort Mutation on GOMT Problem. Sort mutation in Iris refers to converting an original GOMT problem into a related GOMT problem in which the underlying sorts are systematically changed.

Definition 3.1 (Sort Mutated GOMT (SM-GOMT)). *Given a GOMT problem $GO = \langle t, \prec, \varphi \rangle$ over a signature Σ, an SM-GOMT problem is defined as a tuple SM-GO $:= \langle GO, M \rangle$, where $M : \Sigma \to \Sigma'$ is a signature mapping that transforms sorts, function symbols, and constants from Σ to a target signature Σ'.*

Under M, the objective term t, the strict partial order $\prec$, the feasibility formula φ, and any interpretation I of Σ are transformed to $M(t)$, $M(\prec)$, $M(\varphi)$, and $M(I)$, respectively.

*We write $M(t)_J$ to denote the evaluation of the transformed term $M(t)$ under an interpretation J of Σ'. For interpretations J, J' satisfying $M(\varphi)$, we say J **SM-GO-dominates** J', denoted $J <_{SM\text{-}GO} J'$, if and only if $M(t)_J \; M(\prec) \; M(t)_{J'}$.*

Importantly, sort mutation is not required to preserve logical equivalence, i.e., for a GO-consistent interpretation I and its corresponding SM-GO interpretation $M(I)$, we do not strictly require $\varphi \equiv M(\varphi)$. Nonetheless, we identify two key properties of SM-GO that enable sound solver validation:

Algorithm 1: Iris's pseudocode

```
1  Procedure Iris(Seeds, Solver, Theory_src, Theory_tgt)
2      Bugs ← ∅
3      foreach φ in Seeds do
           /* Add objective functions based on grammars                    */
4          φ_t ← ObjFuncArrangement(φ, Theory_src)
           /* Transform formula from source to target theory               */
5          M(φ_t) ← TheoryTrans (φ_t, Theory_src, Theory_tgt)
6          Res ← ApproxValidate(M(φ_t), φ_t)
7          if Violate(Res, Properties) then
8              Bugs ← Bugs ∪ {(M(φ_t), φ_t)}
9  Procedure TheoryTrans(φ_temp, T_src, T_tgt)
       /* Convert variable and function definitions                        */
10     φ_temp ← ConvertDef(φ_temp, T_src, T_tgt)
11     if T_src = Bitvectors then
           /* Add range constraints at random                              */
12         φ_temp ← AddRangeConstraints(φ_temp, T_src, T_tgt)
       /* Traverse the syntax tree of φ_temp                               */
13     foreach term in φ_temp do
14         if term has subterms then
15             foreach subterm in term do
16                 TheoryTrans(subterm, T_src, T_tgt)
17         else if term is a constant then
18             term ← ConvertSort(term, T_src, T_tgt)
19         else if term.op ∈ SpecialOps_{T_src→T_tgt} then
20             term ← ConvertSort(term, T_src, T_tgt)
21     return φ_temp
```

Definition 3.2 (Regulated Consistency). *A sort mutator M satisfies* regulated consistency *if, for any GO-consistent interpretation I (i.e., $I \vDash \varphi$), the transformed interpretation $M(I)$ satisfies $M(\varphi)$.*

Intuitively, regulated consistency ensures that every satisfying assignment of the original problem maps to a satisfying assignment of the transformed problem. Note that the converse need not hold: $M(\varphi)$ may admit solutions with no pre-image under M.

Definition 3.3 (Regulated Domination). *A sort mutator M satisfies* regulated domination *if, for any interpretations I and I' such that I GO-dominates I' ($I <_{GO} I'$), it holds that $M(I)$ SM-GO-dominates $M(I')$ ($M(I) <_{SM\text{-}GO} M(I')$).*

Regulated domination ensures that the relative ordering of objective values is preserved under the transformation. Together with regulated consistency, it guarantees that the structure of the optimization problem is maintained across theories. These two properties are independent: regulated consistency concerns the preservation of feasibility, while regulated domination concerns the preservation of the objective ordering. We call a sort mutator M *regulated* if it satisfies both properties.

Because the transformed problem $M(\varphi)$ may admit solutions outside the image of M, an SM-GO problem may possess a different optimal solution than the original GO problem. Mirroring the definition of a GO-solution described in Definition 2.2, we define an SM-GO solution as follows:

Definition 3.4 (SM-GO Solution). *An interpretation J is an* SM-GO solution *if J satisfies $M(\varphi)$ and no interpretation J' satisfying $M(\varphi)$ SM-GO-dominates J. Equivalently, for all feasible J', $J \leq_{SM\text{-}GO} J'$ (i.e., $M(t)_J \; M(\preceq) \; M(t)_{J'}$). If the optimum is unbounded, no SM-GO solution exists.*

We now state our main theorem, which relates SM-GO solutions to the image of original GO-solutions under a regulated sort mutator.

Theorem 3.1. *Let M be a regulated sort mutator, I be a GO-solution, and I' be any GO-consistent interpretation that is not a GO-solution. If an SM-GO solution J exists, then $M(t)_J \; M(\preceq) \; M(t)_{M(I)}$ and $M(t)_J \; M(\prec) \; M(t)_{M(I')}$.*

Proof. We proceed in three steps.

1. **Regulated Consistency**: By definition, a regulated mutator M satisfies regulated consistency. Because I is GO-consistent ($I \vDash \varphi$), it follows that $M(I) \vDash M(\varphi)$. In other words, $M(I)$ is a feasible solution to the transformed problem.

2. **Regulated Domination**: Assuming the objective values form a total order, because I is a GO-solution and I' is a GO-consistent but suboptimal interpretation, I strictly GO-dominates I' ($I <_{GO} I'$). By regulated domination, this implies $M(I) <_{SM\text{-}GO} M(I')$, which expands to $M(t)_{M(I)} \; M(\prec) \; M(t)_{M(I')}$.

3. **Optimality of** J: Because J is an SM-GO solution and $M(I)$ is feasible (from Step 1), the definition of SM-GO optimality dictates that $J \leq_{SM\text{-}GO} M(I)$, meaning $M(t)_J \; M(\preceq) \; M(t)_{M(I)}$. By the transitivity of the transformed order relations, combining this with Step 2 yields $M(t)_J \; M(\prec) \; M(t)_{M(I')}$.

Therefore, both $M(t)_J \; M(\preceq) \; M(t)_{M(I)}$ and $M(t)_J \; M(\prec) \; M(t)_{M(I')}$ hold. □

Integer to Real Conversion. The conversion from integer to real arithmetic is a quintessential regulated sort mutation. Note that the conversion strategy can be designed in different ways. Here, Iris employs a conservative transformation strategy designed to preserve the original semantics exactly, thereby preventing *semantic drift*, i.e., unintended alterations to the formula's feasible region or optimization landscape. Specifically, Iris's conversion process updates the types of variables and functions to real numbers, and adjusts terms in `assert` commands and optimization goals to align with real arithmetic as specified in the SMT-LIB standard. The conversion proceeds in several steps. First, all integer variables are redefined to have the `Real` sort, and functions returning integers are modified to return real numbers (Line 10). Next, by traversing the formula's syntax tree, terms in `assert` commands and optimization objectives are correspondingly updated (Lines 13–20). Crucially, to maintain strict compliance with the SMT-LIB standard, integer-specific operations such as Euclidean division (`div`) and modulo (`mod`) cannot be naively replaced by real division (`/`). Instead, Iris encodes these operations by composing real arithmetic with the `to_int` function (Lines 19–20). For instance, an integer division term `(div a b)` should be transformed into `(to_real (div (to_int a) (to_int b)))` to ensure that the result remains an integer value within the real domain. Because the underlying variables are promoted to reals, this wrapping enforces discrete evaluation boundaries within the continuous domain, perfectly preserving the original integer semantics. Finally, any pre-existing `to_real` operators are stripped away, as they become redundant. This process continues iteratively until all terms are converted, ensuring that the real arithmetic formula maintains semantics nearly equivalent to the original. The modified formula is then returned (Line 21).

Bit-Vectors to Integers Conversion. The conversion from bit-vectors to integers is another typical regulated sort mutation. In particular, translating bit-vectors to integers requires careful handling of fixed-width semantics. To transform bit-vectors to integers, Iris replaces each bit-vector

variable of width w with an integer variable, and maps bit-vector literals to their unsigned integer equivalents. To accurately capture overflow and underflow behaviors, the translation of predicates is width-aware: unsigned comparisons map directly to integer comparisons, whereas signed predicates and conversions are encoded using a signed-view conditional term, $\text{ite}(x \geq 2^{w-1},\ x - 2^w,\ x)$. Furthermore, to reflect the bounded nature of bit-vectors, Iris can enforce range constraints, such as $0 \leq x < 2^w$, e.g., adding the constraint `(assert (and (>= x 0) (< x 65536)))` for a 16-bit variable. Notably, to increase the diversity of generated test cases, Iris can optionally relax or omit these bounds, yielding a controlled over-approximation compared to the original bit-vector semantics (Lines 11–12). To translate bit-vector operators, Iris employs two distinct strategies: *nonlinear* and *mixed* modes. In *nonlinear* mode, arithmetic bit-vector operators (e.g., `bvadd`, `bvsub`, `bvmul`, `bvudiv`) are compiled into integer arithmetic and wrapped with a modulo 2^w operation. While this fully arithmetizes the formula, it can introduce non-linear integer arithmetic (e.g., non-linear `mod` terms). Alternatively, in `mixed` mode, Iris preserves the original operators by introducing conversion bridges. Specifically, operations are wrapped in bridge operators such as `int_to_bv`, `sbv_to_int`, and `ubv_to_int` to maintain the relationship between the two domains. Either strategy can be selected based on the features supported by the target solver.

3.4 Approximation Property Validation

To validate OMT solvers, Iris leverages the formal properties established above to check the relationship between optimal values across theories and uncover potential bugs. Concretely, given a formula and its sort-mutated counterpart, Iris executes the solver on both versions and inspects the results for unexpected discrepancies in satisfiability or optimality.

Since the sort mutations described above are regulated, two soundness conditions follow directly from the definitions. First, *satisfiability preservation*: if an integer formula φ is satisfiable, then $M(\varphi)$ must also be satisfiable (by regulated consistency). Equivalently, if $M(\varphi)$ is unsatisfiable, then φ must also be unsatisfiable. Second, *optimality ordering*: by Theorem 3.1, the SM-GO optimal value cannot be worse than the image of the original GO optimal value, i.e., $M(t)_J\, M(\preceq)\, M(t)_{M(I)}$. In the concrete case of integer-to-real conversion, this means the real-arithmetic optimum should be at least as good as the integer-arithmetic optimum. Iris checks these two conditions after running the solver on both versions of the formula. Any violation of the expected satisfiability or optimality relationship signals a potential bug in OMT solvers, which can then be further analyzed and reported to developers.

Upon detecting a violation, Iris employs an automated reduction pipeline to streamline bug analysis. This pipeline systematically minimizes the complex, bug-inducing formula while guaranteeing that the underlying solver discrepancy remains reproducible. Unlike standard single-file reduction techniques used in prior work [47], our methodology must preserve the behavioral relationship between the divergent solver executions. To achieve this, we extend CReduce [42] to dynamically regenerate test pairs during the minimization process. For each candidate reduction of the seed formula, the pipeline automatically applies the specified theory transformation, such as integers to reals ($Int \rightarrow Real$) or bit-vectors to integers ($BV \rightarrow Int$), to synthesize a new paired counterpart. It then re-evaluates the solver on both the reduced seed and the newly mutated formula. A reduction step is accepted only if it successfully reproduces the original failure, e.g., an optimality relationship mismatch or a soundness violation. By decoupling the transformation strategies from the property-checking oracle, this framework remains highly extensible across diverse theories and solvers. Ultimately, this dynamic reduction process yields minimal, isolated formulas that can reduce the manual debugging burden when reporting issues to solver developers.

Table 1. Status of bugs found in the solvers.

Status	Z3	OptiMathSAT	Symba	OCAC	Total
Reported	11	10	1	2	24
Confirmed	10	8	1	2	21
Fixed	10	8	1	1	20

Table 2. Bug types among the reported bugs.

Status	Z3	OptiMathSAT	Symba	OCAC	Total
Opt. Correctness	8	4	1	1	14
Crash	3	4	0	1	8
Soundness	0	2	0	0	2

4 Evaluation

This section evaluates the effectiveness of Iris through detailed experiments.

4.1 Evaluation Setup

Research Questions. Our experiments aim to answer the following research questions:

- **RQ1:** Can Iris identify genuine bugs in OMT solvers? (Section 4.2)
- **RQ2:** Does Iris effectively exercise the optimization functionalities of OMT solvers? (Section 4.3)
- **RQ3:** Can Iris complement established testing paradigms for OMT solvers? (Section 4.4)

Targeted Solvers. Our evaluation focuses on four representative OMT solvers: Z3 [10], OptiMathSAT [44], Symba [31], and OCAC [25]. OptiMathSAT is a feature-rich solver that supports optimization over a wide range of theories, including both linear and non-linear arithmetic. However, its closed-source nature limits our ability to perform in-depth analysis. To address this, we also include Z3, a widely adopted open-source solver with over 12,000 GitHub stars, which supports linear arithmetic optimization and offers broad community support. To further diversify our study, we include Symba and OCAC. Symba was originally developed as a standalone tool but is now integrated into Z3 as an alternative optimization engine. OCAC, by contrast, is supposed to be integrated into cvc5 [5], a leading SMT solver widely used in both academia and industry.

Test Seeds. Iris is designed to generate test instances for OMT solvers based on SMT-LIB formulas. Due to the decades-long history of the SMT community, there is a wealth of SMT-LIB instances available for testing and benchmarking SMT solvers. In the experiments, we employed formulas generated by the existing SMT solver testing tool ET [52] as starting points. These formulas are typically small in size and cover a wide range of theories, including linear arithmetic. Their small size facilitates the inspection and reduction of bug-revealing cases during the bug-hunting campaign. In total, we utilized one million formulas generated by ET as seeds for our experiments.

Environment. Experiments were conducted on a machine with an Intel Xeon Gold-5117 processor (48 cores, 128 GB RAM), running an Ubuntu 20.04 (64-bit) Docker container. Additionally, we set a timeout of 30 seconds for each solver invocation to prevent excessive execution times.

4.2 RQ1: Bug Detection

The goal of RQ1 is to determine whether Iris can identify genuine bugs in OMT solvers, providing insight into its effectiveness. To this end, we applied Iris to the latest trunk versions of Z3, Symba, and OCAC, which reduces the chance of re-discovering known issues. Since OptiMathSAT is not open-source, we contacted its developers and obtained permission to test their solver using their provided nightly builds. To ensure responsible reporting, we followed a careful disclosure process. Specifically, each bug was submitted individually to the solver developers and classified by type. When multiple issues of the same category were found, we held back subsequent reports until the

developers addressed the initial one. This procedure minimized duplicate reports and facilitated effective communication with the solver teams. The issues detected by Iris fall into three major categories:

- **Optimization Correctness Bugs:** Instances where the solver produces inaccurate optimization results, such as reporting an incorrect optimal value or failing to identify a valid solution.
- **Soundness Bugs:** These are cases where the solver returns incorrect satisfiability results, such as reporting `sat` for unsatisfiable formulas or `unsat` for satisfiable ones.
- **Crashes:** These occur when the solver terminates unexpectedly, for instance, due to segmentation faults or assertion failures.

After several days of validation, Iris identified 306 potential issues spanning the three categories above. For optimization correctness and soundness bugs, each potential issue corresponds to a pair of formulas whose solver outputs violate the cross-theory approximation property. To distinguish optimization errors from logical errors, we remove the optimization objectives and rerun the same constraints. If the solver still produces a valid satisfiability result but reports an incorrect optimum, we classify the issue as an optimization correctness bug. If adding objectives changes the satisfiability result, we classify it as a soundness bug. Crashes are classified separately based on their runtime behavior. This classification captures the user-visible failure mode; the eventual fix may involve the optimization engine, the underlying SMT engine, or their interaction.

For further triage, we group crashes according to traceback or assertion signatures, and group optimization correctness and soundness bugs according to the solver, theory pair, and violated property. This process organizes the 306 potential issues into 21 heuristic groups. These groups are used only to prioritize investigation and do not imply that cases within a group share the same root cause. We initially report one representative case from each prioritized group and revisit the remaining cases after developer fixes become available. If the remaining cases continue to fail after the fix, we report them as additional issues. At the time of this study, this process had resulted in 24 reported bugs. Table 1 summarizes these reported issues.

In total, we reported 24 bugs to the respective solver developers, of which 21 have been confirmed and 20 have been resolved. The remaining three bugs are still under investigation and pending resolution. As illustrated in Table 2, the majority of these reports (14 out of 24) involve optimization correctness issues, demonstrating the specialized capability of Iris in identifying optimization-related bugs in OMT solvers. Such bugs are particularly critical as they are difficult to detect yet can result in subtle errors in the solver's output. Furthermore, we identified two soundness bugs that led to incorrect satisfiability results, directly stemming from flaws in the underlying SMT solver. The effectiveness of our approach is driven by the two designed transformation strategies. Specifically, 17 out of the 24 bugs were identified through integer-to-real transformations, while the remaining issues were uncovered via bit-vector-to-integer transformations. The success of both methods suggests they are complementary and effective at exposing bugs in OMT solvers. Importantly, although we observed several duplicate reports during manual inspection, we did not identify false positives during our bug-hunting campaign, which indicates that the approach is reliable for identifying genuine solver issues.

Bug Lifespan. To further assess the significance of the identified bugs, we analyze when each of the 20 fixed bugs was introduced. Specifically, for each bug-revealing test case, we track the solver's behavior across different releases to determine the earliest version affected. This analysis reveals the lifespan of each bug, measuring the time between its introduction and resolution. Figure 6 shows the distribution of bugs over time. Our findings indicate that a bug remained undetected

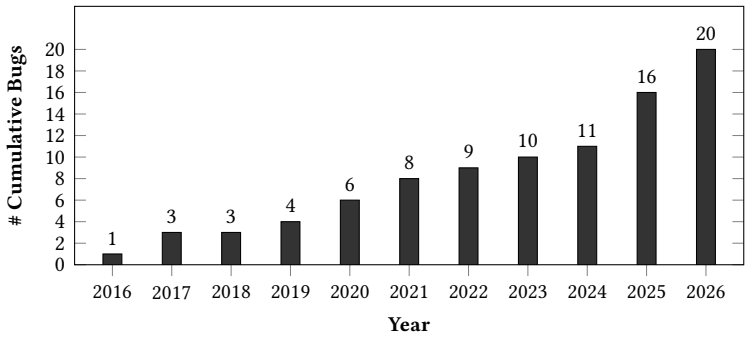


Fig. 6. Lifespan of the fixed bugs in the OMT solvers.

for nearly a decade, while approximately half of the fixed bugs persisted for at least five years. These results demonstrate Iris's effectiveness in uncovering long-standing bugs in OMT solvers.

4.3 RQ2: Ability to Exercise OMT Solvers

The goal of *RQ2* is to evaluate whether Iris effectively exercises OMT solvers. To this end, we first measure the code coverage achieved by Iris during validation and then complement this measure with a known-bug rediscovery experiment.

Code Coverage. In this experiment, we focus on Z3 version 4.16, the latest release at the time of our study. Z3 is open source, allowing us to collect detailed coverage information using `gcov` [21]. OptiMathSAT is excluded because it is closed source and cannot be instrumented for coverage measurement. We do not separately analyze Symba because it is integrated into Z3, and we exclude OCAC because its instability (e.g., frequent crashes) prevents reliable coverage collection.[1] In this experiment, we also use the test cases generated by ET as seed inputs. ET [52] is one of the most recent SMT solver testing techniques and generates the seed formulas used in our evaluation of RQ1. We use `gcov` to measure line coverage, with particular attention to the optimization-related implementation in Z3's "`src/opt`" directory. We compare three configurations: the original ET seeds without optimization objectives, Iris-w/oG, which selects existing well-typed seed terms as objectives, and the complete Iris, which additionally generates objectives using grammars. For each configuration, we evaluate ten cumulative seed budgets from 1,000 to 10,000 formulas, increasing by 1,000 formulas at each step. We repeat each measurement 10 times and report the average coverage. Figure 7 presents the coverage trends over the evaluated seed range.

The figure shows that both Iris and its variant consistently achieve higher coverage than the original ET seeds, demonstrating that adding optimization objectives is essential for reaching optimization-related code. Furthermore, Iris consistently outperforms Iris-w/oG, showing that grammar-based objective generation explores additional optimization behaviors beyond simply reusing existing terms. The coverage curves gradually flatten across the evaluated seed range, suggesting diminishing returns from generating more seeds with the same distribution.

Note that the overall code coverage remains below 30%, which is expected under our evaluation setting. Our experiments only consider SMT-LIB command-line inputs with integer and bit-vector formulas, leaving other theories, interfaces, and optional features unexplored. Moreover, the coverage percentages are measured against Z3's approximately 600K-line codebase [54], where even small percentage changes may represent substantial additional behavior. More importantly, the

[1] https://github.com/fuqi-jia/cdcl_ocac/issues/3

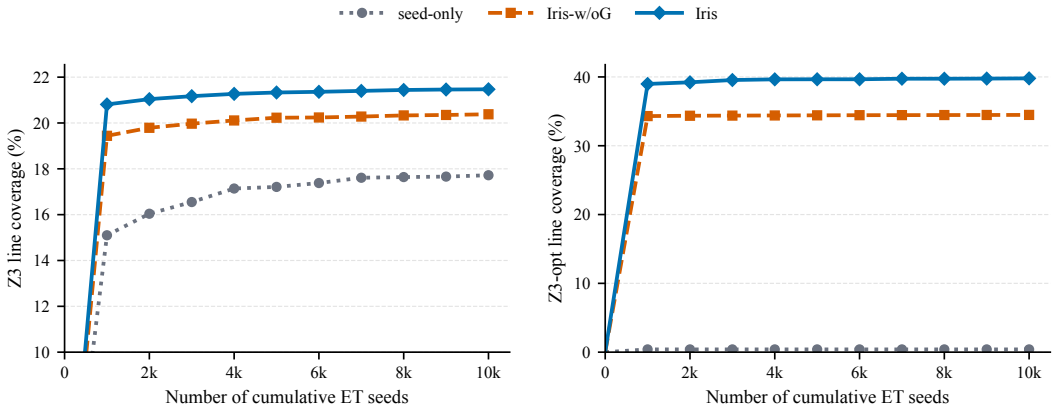


Fig. 7. Line coverage over cumulative seed count for the complete Z3 codebase (left) and its optimization components, Z3-opt (right). Iris-w/oG selects existing well-typed seed terms as objectives, and Iris uses grammar-generated objectives.

goal of Iris is not to maximize coverage, but to provide an effective oracle for validating OMT solvers. Therefore, coverage serves as evidence that generated objectives successfully activate optimization functionality rather than as an optimization target itself.

Bug-finding Capability. Code coverage measures how broadly a tool explores the solver, but it does not directly indicate whether the explored behaviors can reveal real bugs. Therefore, we further evaluate the bug-finding capability of Iris and Iris-w/oG by running both configurations for 24 hours on Z3 4.14.0 and OptiMathSAT 1.7.4 with the same seed corpus. The time budget follows prior work [52], and the solver versions correspond to the starting points of our bug-finding campaigns in RQ1. For each configuration, we count the distinct known bugs rediscovered during the campaign and measure their overlap across the two solvers. To identify unique bugs, we use different criteria for the two solvers. For Z3, we replay each failure across the revision history and use binary search to locate its *correcting commit*, after which the failure no longer occurs [14]. Failures with the same correcting commit are considered duplicates. Since OptiMathSAT is closed source, we instead replay each failure across available solver versions and represent its behavior using a Boolean occurrence vector, where each entry indicates whether the failure appears in a particular version [32]. Failures with identical vectors are conservatively treated as duplicates, while different vectors are counted as distinct bugs.

Iris rediscovers five unique known bugs, which are all optimization correctness issues, while Iris-w/oG finds three. Note that all bugs found by Iris-w/oG are also detected by Iris. Thus, grammar-based objective generation enables Iris to expose two additional known bugs beyond the simpler strategy of reusing existing objective terms, supporting the coverage results that it reaches additional optimization functionality.

Overall, these results show that Iris can effectively exercise OMT solver optimization features and that grammar-based objective generation improves both exploration and bug detection compared with simpler objective construction strategies.

4.4 RQ3: Complementarity to Established Paradigms

To evaluate the unique utility of Iris, we compare it with two established testing paradigms: *differential testing* and direct *solution verification*. Differential testing runs multiple solvers on the

Table 3. Retrospective examination of the 11 optimization-correctness bugs.

	Differential Testing		Solution Verification	
Solver	*Detected*	*Missed*	*Detected*	*Missed*
OptiMathSAT	1	3	3	1
Z3	3	4	2	5
Total	**4**	**7**	**5**	**6**

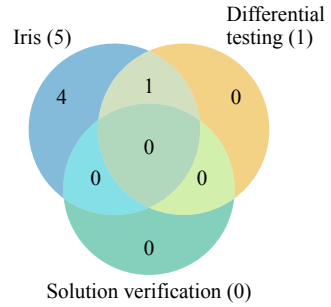


Fig. 8. Unique bugs exposed during 24-hour runs.

same formula and checks whether their optimization results disagree, whereas solution verification checks whether a solver's reported result is feasible and whether a strictly better solution exists. We study these paradigms from two perspectives: whether they can expose previously reported bugs when given the triggering inputs, and how their end-to-end bug-finding results overlap under a fixed time budget.

Examination of Reported Bugs. For this analysis, we focus specifically on optimization correctness rather than other bug types, such as crashes, because crashes are not a meaningful differentiator for an oracle-based technique and are typically detectable by any execution-based testing method. Specifically, we conduct our evaluation using the 11 fixed optimization-correctness bugs from RQ1 as subjects. We use Z3 and OptiMathSAT as reference solvers for differential testing, as they currently offer the most comprehensive support for various theories. For each bug, we provide the reported bug-triggering formula to both reference solvers and monitor for inconsistent outputs. To ensure a rigorous comparison, we utilize the most recent stable releases (Z3 4.16 and OptiMathSAT 1.7.4). In cases where a reference solver fails to terminate or returns `unknown`, we consider the bug undetectable via differential testing for that specific input. For solution verification, we check the feasibility of the reported solution and query whether a strictly better solution exists; a bug is considered detected when either check refutes the reported optimization result. Table 3 shows that differential testing detects 4 of the 11 bugs, while solution verification detects 5; neither paradigm can expose all of the reported correctness failures. This limitation is primarily driven by the uneven support for various mathematical theories across different solver engines. Although SMT-LIB provides a common language for base SMT solving, optimization support is less uniform across implementations [51], making direct cross-solver comparisons fragile for some OMT fragments. For example, while OptiMathSAT supports optimization over non-linear integer arithmetic, solvers such as Z3, Symba, and OCAC do not yet implement these features, precluding direct comparison for formulas in those domains. Solution verification is likewise incomplete for results involving unattained optima or unbounded limits, for which there may be no concrete optimal model or finite better-solution query. Thus, the table measures the ability of each oracle to recognize known failures when supplied with their triggering inputs.

Unique Known Bug Rediscovery. We next compare Iris, *differential testing*, and *solution verification* under the same 24-hour time budget, solver versions, seed corpus, and bug identification protocol. For each approach, we use the same seeds and count the number of unique known bugs

rediscovered. Following the protocol in RQ2, we identify unique Z3 bugs using the correcting-commit method [14] and unique OptiMathSAT bugs using the version-vector method [32]. Figure 8 summarizes the results. Iris rediscovers five unique known bugs, four of which are found exclusively by Iris. Differential testing rediscovers one bug, which is also detected by Iris, while solution verification finds no bugs within the 24-hour budget. Table 3 evaluates whether each oracle can recognize known failures when given their triggering inputs, whereas this experiment evaluates the end-to-end ability to discover such failures within a fixed time budget. Together, these two evaluations provide complementary views of oracle effectiveness: one measures the power of the oracle itself, while the other measures its practical bug-finding capability when combined with input generation.

4.5 Bug Samples

To better illustrate the bugs identified by Iris, we examine a selection of reported issues in detail. Figure 9 presents six cases: two adapted from OptiMathSAT and four from Z3. These examples showcase distinct solver bugs, highlighting their causes and resolutions.

Figure 9a. shows a Z3 bug that reports incorrect finite and infinite optima for a problem with three objectives. For the simplified example, Z3 returns 278, $+\infty$, and $-\infty$ as the optimal values of the three objectives, although none of the objectives should be unbounded. The finite result 278 can be disproved by finding a better solution, but directly validating an incorrect $+\infty$ or $-\infty$ result is more challenging because no concrete value can exceed these bounds. Such cases require proving boundedness rather than simply checking a better candidate solution. In contrast, Iris's cross-theory oracle detects the inconsistency without requiring an explicit optimal model. The Z3 developers fixed this issue by improving the handling of multiple objectives, ensuring that each objective is optimized independently and that unbounded objectives are reported correctly.

Figure 9b. exposes a critical correctness bug in OptiMathSAT that affects satisfiability results. Specifically, the solver returns `unsat` for the instance in Figure 9b but deems the same constraint `sat` when the optimization objective is removed. This inconsistency reveals a fundamental flaw in how the solver handles satisfiability when optimization is involved, dramatically undermining its reliability.

Figure 9c. reveals a correctness issue in Z3, where the solver delivers inaccurate optimization results. Specifically, when handling multiple objective functions separately, Z3 incorrectly reports the minimum value of the second objective, `(- a 0)`, as 0. However, converting the formula to integer arithmetic reveals the true minimum as $-\infty$, exposing a contradiction. Developers have since improved Z3's handling of linear real arithmetic to address this bug.

Figure 9d. highlights a similar correctness issue in OptiMathSAT, also resulting in erroneous optimization outcomes. The developers attribute this subtle bug to a recent update that impaired the solver's ability to process modulo operations correctly. Note that differential testing failed to detect this issue, as Z3 does not support optimization over non-linear integer arithmetic, causing it to stall when evaluating the same formula. Thus, there is no effective reference for comparison.

Figure 9e. highlights an inconsistency in Z3's optimization engine regarding scaled objective functions. Interestingly, while Z3 correctly determines the minimum value of the variable a, it incorrectly reports a minimum when minimizing a positive scalar multiple of the same variable ($3a$), as well as the original integer formula. This discrepancy likely stems from premature termination in the optimization loop, as indicated by the developers' analysis. To resolve this, developers appear

```
(declare-const a Int)
(declare-const b Int)
(assert (and (>= b (- 166))
             (<= a (- 166))
 (>= (* 5 a) (+ (* 9 b) 178))))
(maximize (- (+ a b)))
(maximize b) (minimize a)
```

(a) Simplified Z3 case producing incorrect finite and infinite optima.

```
(declare-const a Real)
(declare-const b Real)
(assert (> b (- 1 (- a 0))))
(minimize (- 1 (- a 0)))
(minimize (- a 0))
(check-sat)
(get-objectives)
```

(c) Correctness issue detected by Iris that leads to incorrect optimization results in Z3.

```
(declare-const a Real)
(declare-const b Real)
(assert (xor (= 0 b)
       (> (* (- a) 50) 3)))
(minimize (* 3 a))
(check-sat)
(get-objectives)
```

(e) A simplified test case produced by Iris that triggers an incorrect optimization result in Z3.

```
(declare-const a Real)
(declare-const b Real)
(assert (and (>= (- 0 b) b)
         (= 1 a)))
(maximize (/ b a))
(check-sat)
```

(b) Correctness issue detected by Iris that leads to incorrect satisfiability results in OptiMathSAT.

```
(declare-const a Int)
(declare-const b Int)
(assert (and (>= 0 b)
         (= (mod a b) b)))
(minimize (mod a b))
(check-sat)
(get-objectives)
```

(d) Correctness issue detected by Iris that leads to incorrect optimization results in OptiMathSAT.

```
(declare-const a (_ BitVec 8))
(declare-const b (_ BitVec 32))
(declare-const d (_ BitVec 32))
(minimize (bvmul d #x1FF4EAF0))
(minimize b)
(maximize (bvneg (bvmul #xE8 a)))
(get-objectives)
```

(f) Original formula produced by Iris whose mutant exposes an optimization correctness issue in Z3.

Fig. 9. Selected bug samples in Z3 and OptiMathSAT.

to have introduced a fallback mechanism that uses previously discovered solutions to help the solver resume its search.

Figure 9f. presents a bit-vector formula whose mutant exposes a correctness issue in Z3's optimization engine. Specifically, when maximizing the objective corresponding to the 8-bit objective `(bvneg (bvmul #xE8 a))`, Z3 incorrectly reports 0 rather than the true maximum. The issue arises because the solver, when juggling multiple objectives, inadvertently carries over stale data from one objective to the next. Developers resolved this by capturing a "baseline" valid state before optimization begins and using it to ensure each subsequent objective is calculated from a reliable, consistent starting point.

5 Discussion

In this section, we further discuss the experimental results, threats to validity, ethical considerations, and limitations. We also discuss how to enhance this work in the future.

5.1 Threats to Validity

Our evaluation is subject to several threats, which we summarize below. First, the effectiveness of Iris may depend on the quality of the initial test cases. Poorly chosen seeds could limit its ability to uncover issues. To mitigate this, we used one million formulas generated by ET, one of the most advanced SMT solver testing tools, as starting points. These formulas are broadly representative of practical SMT-LIB instances and help ensure that bug-revealing cases remain manageable in size. Second, the generation of objective functions relies on predefined grammars that introduce randomness, which may affect the consistency of evaluation results. To reduce this risk, we conducted multiple trials for relevant experiments and report average results, smoothing out fluctuations due to chance. Third, the choice of subject solvers may affect the generalizability of our findings. We selected four state-of-the-art OMT solvers, i.e., Z3, OptiMathSAT, Symba, and OCAC, covering both open-source and proprietary systems. This diversity provides a comprehensive perspective on the effectiveness of Iris. Additionally, determining the number of distinct known bugs exposed in our experiments is non-trivial. To mitigate this threat, we identified bugs using bug-fixing commits and Boolean occurrence vectors across solver versions, following established techniques from prior work [14, 32]. Finally, in *RQ3* we assessed differential testing using bug-triggering formulas identified by Iris. While this cannot entirely rule out the possibility that differential testing might also find these bugs, it provides a practical way to estimate its ability. This practice is also consistent with prior work [55].

5.2 Ethical Considerations

This study was conducted with careful attention to ethical principles. Because OMT solvers, as well as the underlying SMT solvers, can be used in safety-critical systems, improving their correctness provides clear benefits to both researchers and practitioners. Our goal is to strengthen solver reliability without creating an unnecessary burden for developers or users. We adopted a responsible reporting process to ensure that our findings were constructive. Each bug-triggering case was reviewed, minimized, and validated before submission. We reported issues individually and paused further reports of the same type until earlier ones were resolved, helping developers address them efficiently. We maintained collaborative communication with solver developers throughout the study. We obtained consent before testing their systems and disclosed issues in a responsible manner. Developers responded positively and appreciated the clarity of our reports. Overall, this work contributes to more reliable OMT solvers and supports their safe use in practice.

5.3 Limitations

While Iris has demonstrated effectiveness in identifying bugs in OMT solvers, it also has several limitations. First, the current implementation supports transformations among integer, real, and bit-vector theories. We focus on these theories because they cover the optimization features supported by the evaluated solvers and are widely used in practice. Supporting additional theories requires more than extending the syntax: the target theories must have meaningful relationships, such as domain inclusion, and the transformation must preserve the expected relationships between optimization bounds. Floating-point arithmetic is a promising but challenging extension because it shares connections with real arithmetic while introducing features such as rounding

modes that require careful handling. String constraints are more difficult because only limited operations, such as `str.len`, naturally relate to integer arithmetic, while full string semantics do not directly fit our current validation framework. Consequently, theories without a natural cross-theory relationship are currently beyond the scope of Iris.

Second, Iris is designed as a black-box testing framework, enabling it to validate closed-source solvers such as OptiMathSAT and execute efficiently without requiring internal solver information. However, this design prevents Iris from directly using internal coverage information to guide test generation. For open-source solvers, incorporating coverage-guided strategies could further improve exploration efficiency and help reach deeper solver behaviors.

5.4 Future Work

In the future, we plan to extend Iris in several directions. Specifically, our current evaluation focuses on four advanced OMT solvers: Z3, OptiMathSAT, Symba, and OCAC. Although these solvers represent the major available OMT implementations, we plan to extend the evaluation to additional solvers, including BCLT [29] and the upcoming official release of cvc5[2]. We also plan to support more advanced optimization features [51] to enable broader evaluation of solver correctness and reliability in the future.

6 Related Work

OMT Solver. OMT solvers are essential software systems that are used to solve constrained optimization problems. Over the past decade, they have advanced significantly and are now increasingly applied in real-world systems. Since 2016, OMT solvers have been adopted by the security, systems, and programming languages communities, with applications in deterministic network scheduling [17, 40], differential privacy proofs [28, 58], and network configuration synthesis [8]. More recently, their capabilities have expanded to a broader range of domains, including fuzzing [23], side-channel analysis [19], formal verification [45], and quantum compiler optimization [38, 39]. Given these high-stakes applications, ensuring the correctness and reliability of OMT solvers is essential for their safe and trustworthy deployment. Modern OMT solvers integrate optimization techniques directly into the decision-making procedures of the underlying SMT solvers. This tight integration enables more efficient search strategies and better handling of complex constraints [50]. As a result, validating OMT solvers not only helps detect bugs specific to optimization but can also uncover deeper issues in the underlying SMT solvers. For example, during our study, two correctness bugs reported by Iris led to incorrect satisfiability results, pointing to fundamental bugs in the base SMT solver. Furthermore, crashes in the OMT engine of solvers such as Z3 highlight potential risks. If left unchecked, such issues could be exploited to attack systems that depend on these solvers [1–3]. Thus, validating OMT solvers serves a dual purpose: it safeguards their correctness in critical applications and strengthens the robustness of the SMT solvers beneath them. This dual impact ultimately enhances the reliability of the broader solver ecosystem [27, 41, 46, 57].

Solver Testing. While there is no prior work dedicated to validating OMT solvers, significant progress has been made in testing related automated reasoning systems such as SMT solvers. Early work in SMT solver testing includes FuzzSMT, a generation-based fuzzing technique proposed by Brummayer and Biere [12]. Subsequent approaches have explored mutation-based fuzzing to expose bugs in SMT solvers [34, 48, 49, 53, 54, 59]. Beyond SMT, Carvalho et al. [13] introduced a testing framework for Linear Temporal Logic (LTL) solvers. However, none of these approaches address the unique challenges posed by OMT solvers, which go beyond satisfiability checking

[2] https://github.com/cvc5/cvc5/issues/10007

to compute optimal solutions under constraints. OMT solvers are increasingly used in domains such as quantum computing [39], heterogeneous systems [18], and program analysis [56], where correctness is critical. Our research addresses this critical gap by introducing a novel validation technique tailored for OMT solvers, specifically designed to identify optimization-related bugs.

Metamorphic Testing. Metamorphic Testing (MT) is a principled software testing paradigm designed to address the test oracle problem [15]. MT has been applied in a variety of domains, including compilers [9, 30], debuggers [32], database systems [26], and protocol implementations [11]. Each of these techniques depends on designing appropriate metamorphic relations (MRs) that define how outputs should obey expected relationships under specific transformations of the input data. The effectiveness and innovation of MT hinge on the design of suitable *metamorphic relations* (MRs), as indicated by the literature [16], which define expected relationships between the outputs of multiple, systematically transformed versions of the same input. In this work, we introduce a novel MT technique tailored to OMT solvers. Our approach, called *cross-theory approximation*, transforms a given formula into a semantically related version expressed in a different logical theory. This transformation induces a predictable relationship between the optimal values of the original and transformed problems, which serves as our metamorphic relation. This technique is theory-aware and does not require external reference outputs. Any deviation from the expected relationship between solutions reveals a potential correctness issue in the solver. Our evaluation demonstrates that this approach can effectively detect bugs in advanced OMT solvers.

7 Conclusion

This paper has introduced Iris, an innovative approach for validating OMT solvers based on cross-theory approximation. The key idea behind Iris is to utilize the inherent relationships between the solution spaces of distinct logical theories to detect bugs in OMT solvers. To achieve this, we carefully devise theory-specific transformations and cross-theory approximation strategies to generate formula variants with nearly equivalent semantics across different theories. By comparing the optimal solutions of these transformed formulas, we can identify potential bugs in the OMT solvers. We have engineered Iris as a practical tool and applied it to the leading OMT solvers, e.g., Z3 and OptiMathSAT. Our evaluation results demonstrate that Iris is effective in detecting bugs in OMT solvers, including correctness issues. Specifically, Iris has successfully identified 24 genuine bugs in the solvers, 20 of which have been fixed by the developers. Moreover, Iris has demonstrated its capability to exercise the optimization features of the solvers.

Data-Availability Statement

The source code of Iris is publicly available at https://github.com/merlin-cs/Iris.

Acknowledgments

We thank the anonymous reviewers for their valuable and constructive feedback. We also thank the Z3 and OptiMathSAT development teams, in particular, Nikolaj Bjørner, Gabriele Masina, and Roberto Sebastiani, for their support and for carefully investigating the bugs we reported. We appreciate Han Zhou's help in improving this work. This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, 62472215, and 62132020), the Jiangsu Natural Science Foundation (Grant BK20231402), the NJU-Huawei Joint Innovation Lab, the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), the "111 Center" (No. B26023), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

References

[1] 2020. Use-After-Free in Z3 (pdd_simplifier.cpp) before 4.8.8. https://nvd.nist.gov/vuln/detail/CVE-2020-19725 Use-after-free vulnerability in Z3 before 4.8.8; causes segmentation faults or potential arbitrary code execution.

[2] 2024. Improper Input Validation in CVC5 Solver v1.1.3 via crafted SMT2 input. https://nvd.nist.gov/vuln/detail/CVE-2024-37794 Improper input validation in CVC5 1.1.3 via crafted SMT2 input, Denial of Service (CVSS 3.1 base score 7.5 HIGH).

[3] 2024. Segmentation Fault in CVC5 Solver v1.1.3 via malformed SMT-LIB input. https://nvd.nist.gov/vuln/detail/CVE-2024-37795 Segmentation fault in CVC5 1.1.3 via crafted SMT-LIB input ('set-logic'), Denial of Service (CVSS 3.1 base score 7.5 HIGH).

[4] Jakob Bach. 2025. Subgroup Discovery with Small and Alternative Feature Sets. *Proc. ACM Manag. Data* 3, 3, Article 221 (June 2025), 27 pages. doi:10.1145/3725358

[5] Haniel Barbosa, Clark W. Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 13243)*, Dana Fisman and Grigore Rosu (Eds.). Springer, 415–442. doi:10.1007/978-3-030-99524-9_24

[6] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2022. *The satisfiability modulo theories library (SMT-LIB).* Retrieved 2022-08-18 from http://smtlib.cs.uiowa.edu/

[7] Clark W. Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. 2021. Satisfiability Modulo Theories. In *Handbook of Satisfiability - Second Edition*, Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh (Eds.). Frontiers in Artificial Intelligence and Applications, Vol. 336. IOS Press, 1267–1329. doi:10.3233/FAIA201017

[8] Ryan Beckett, Ratul Mahajan, Todd D. Millstein, Jitendra Padhye, and David Walker. 2017. Network configuration synthesis with abstract topologies. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 437–451. doi:10.1145/3062341.3062367

[9] Lukas Bernhard, Nico Schiller, Moritz Schloegel, Nils Bars, and Thorsten Holz. 2024. DarthShader: Fuzzing WebGPU Shader Translators & Compilers. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM, 690–704. doi:10.1145/3658644.3690209

[10] Nikolaj S. Bjørner, Anh-Dung Phan, and Lars Fleckenstein. 2015. *vZ* - An Optimizing SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems - 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 9035)*, Christel Baier and Cesare Tinelli (Eds.). Springer, 194–199. doi:10.1007/978-3-662-46681-0_14

[11] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy, SP 2014, Berkeley, CA, USA, May 18-21, 2014*. IEEE Computer Society, 114–129. doi:10.1109/SP.2014.15

[12] Robert Brummayer and Armin Biere. 2009. Fuzzing and Delta-Debugging SMT Solvers. In *Proceedings of the 7th International Workshop on Satisfiability Modulo Theories* (Montreal, Canada) *(SMT '09)*. Association for Computing Machinery, New York, NY, USA, 1–5. doi:10.1145/1670412.1670413

[13] Luiz Carvalho, Renzo Degiovanni, Maxime Cordy, Nazareno Aguirre, Yves Le Traon, and Mike Papadakis. 2024. SpecBCFuzz: Fuzzing LTL Solvers with Boundary Conditions. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 123:1–123:13. doi:10.1145/3597503.3639087

[14] Junjie Chen, Wenxiang Hu, Dan Hao, Yingfei Xiong, Hongyu Zhang, Lu Zhang, and Bing Xie. 2016. An empirical comparison of compiler testing techniques. In *Proceedings of the 38th International Conference on Software Engineering, ICSE 2016, Austin, TX, USA, May 14-22, 2016*, Laura K. Dillon, Willem Visser, and Laurie A. Williams (Eds.). ACM, 180–190. doi:10.1145/2884781.2884878

[15] Tsong Yueh Chen, S. C. Cheung, and Siu-Ming Yiu. 2020. Metamorphic Testing: A New Approach for Generating Next Test Cases. *CoRR* abs/2002.12543 (2020). arXiv:2002.12543 https://arxiv.org/abs/2002.12543

[16] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Comput. Surv.* 51, 1 (2018), 4:1–4:27. doi:10.1145/3143561

[17] Silviu S. Craciunas, Ramon Serna Oliver, Martin Chmelík, and Wilfried Steiner. 2016. Scheduling Real-Time Communication in IEEE 802.1Qbv Time Sensitive Networks. In *Proceedings of the 24th International Conference on Real-Time Networks and Systems, RTNS 2016, Brest, France, October 19-21, 2016*, Alain Plantec, Frank Singhoff, Sébastien Faucou, and Luís Miguel Pinho (Eds.). ACM, 183–192. doi:10.1145/2997465.2997470

[18] Ismet Dagli and Mehmet E. Belviranli. 2024. Shared Memory-contention-aware Concurrent DNN Execution for Diversely Heterogeneous System-on-Chips. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2024, Edinburgh, United Kingdom, March 2-6, 2024*, Michel Steuwer, I-Ting Angelina Lee, and Milind Chabbi (Eds.). ACM, 243–256. doi:10.1145/3627535.3638502

[19] Ferhat Erata, Ruzica Piskac, Víctor Mateu, and Jakub Szefer. 2023. Towards Automated Detection of Single-Trace Side-Channel Vulnerabilities in Constant-Time Cryptographic Code. In *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023*. IEEE, 687–706. doi:10.1109/EUROSP57164.2023.00047

[20] Ivan Gavran, Eva Darulova, and Rupak Majumdar. 2020. Interactive synthesis of temporal specifications from examples and natural language. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 201:1–201:26. doi:10.1145/3428269

[21] Gcov. 2021. Using the GNU Compiler Collection:GCov. https://gcc.gnu.org/onlinedocs/gcc/Gcov.html.

[22] Sankha Narayan Guria, Niki Vazou, Marco Guarnieri, and James Parker. 2022. ANOSY: approximated knowledge synthesis with refinement types for declassification. In *PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, Ranjit Jhala and Isil Dillig (Eds.). ACM, 15–30. doi:10.1145/3519939.3523725

[23] Heqing Huang, Peisen Yao, Rongxin Wu, Qingkai Shi, and Charles Zhang. 2020. Pangolin: Incremental Hybrid Fuzzing with Polyhedral Path Abstraction. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*. IEEE, 1613–1627. doi:10.1109/SP40000.2020.00063

[24] Ruyi Ji, Jingjing Liang, Yingfei Xiong, Lu Zhang, and Zhenjiang Hu. 2020. Question selection for interactive program synthesis. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 1143–1158. doi:10.1145/3385412.3386025

[25] Fuqi Jia, Yuhang Dong, Rui Han, Pei Huang, Minghao Liu, Feifei Ma, and Jian Zhang. 2025. A Complete Algorithm for Optimization Modulo Nonlinear Real Arithmetic. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, Toby Walsh, Julie Shah, and Zico Kolter (Eds.). AAAI Press, 11255–11263. doi:10.1609/AAAI.V39I11.33224

[26] Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 821–835. https://www.usenix.org/conference/osdi24/presentation/jiang

[27] Johannes Krupp and Christian Rossow. 2018. teEther: Gnawing at Ethereum to Automatically Exploit Smart Contracts. In *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*, William Enck and Adrienne Porter Felt (Eds.). USENIX Association, 1317–1333. https://www.usenix.org/conference/usenixsecurity18/presentation/krupp

[28] Martin Kucera, Petar Tsankov, Timon Gehr, Marco Guarnieri, and Martin T. Vechev. 2017. Synthesis of Probabilistic Privacy Enforcement. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM, 391–408. doi:10.1145/3133956.3134079

[29] Daniel Larraz, Albert Oliveras, Enric Rodríguez-Carbonell, and Albert Rubio. 2014. Minimal-Model-Guided Approaches to Solving Polynomial Constraints and Extensions. In *Theory and Applications of Satisfiability Testing - SAT 2014 - 17th International Conference, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 14-17, 2014. Proceedings (Lecture Notes in Computer Science, Vol. 8561)*, Carsten Sinz and Uwe Egly (Eds.). Springer, 333–350. doi:10.1007/978-3-319-09284-3_25

[30] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT Compilers via Compilation Space Exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 66–79. doi:10.1145/3600006.3613140

[31] Yi Li, Aws Albarghouthi, Zachary Kincaid, Arie Gurfinkel, and Marsha Chechik. 2014. Symbolic optimization with SMT solvers. In *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014*, Suresh Jagannathan and Peter Sewell (Eds.). ACM, 607–618. doi:10.1145/2535838.2535857

[32] Giuseppe Antonio Di Luna, Davide Italiano, Luca Massarelli, Sebastian Österlund, Cristiano Giuffrida, and Leonardo Querzoni. 2021. Who's debugging the debuggers? exposing debug information bugs in optimized binaries. In *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems,*

Virtual Event, USA, April 19-23, 2021, Tim Sherwood, Emery D. Berger, and Christos Kozyrakis (Eds.). ACM, 1034–1045. doi:10.1145/3445814.3446695

[33] Pingchuan Ma, Zhaoyu Wang, Zhenlan Ji, Zongjie Li, Ao Sun, and Shuai Wang. 2025. Guardrail: Automated Integrity Constraint Synthesis From Noisy Data. *Proc. ACM Manag. Data* 3, 4, Article 248 (Sept. 2025), 26 pages. doi:10.1145/3749166

[34] Muhammad Numair Mansur, Maria Christakis, Valentin Wüstholz, and Fuyuan Zhang. 2020. Detecting critical bugs in SMT solvers using blackbox mutational fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 701–712. doi:10.1145/3368089.3409763

[35] Gabriele Masina and Roberto Sebastiani. 2025. Exploiting Partial-Assignment Enumeration in Optimization Modulo Theories. In *Frontiers of Combining Systems - 15th International Symposium, FroCoS 2025, Reykjavik, Iceland, September 29 - October 1, 2025, Proceedings (Lecture Notes in Computer Science, Vol. 15979)*, René Thiemann and Christoph Weidenbach (Eds.). Springer, 117–134. doi:10.1007/978-3-032-04167-8_7

[36] Microsoft Research. 2015. *Z3 Issue Tracking System.* Retrieved 2025-5-1 from https://github.com/Z3Prover/z3/issues/

[37] Sadanand Modak, Noah Tobias Patton, Isil Dillig, and Joydeep Biswas. 2025. SYNAPSE: SYmbolic Neural-Aided Preference Synthesis Engine. In *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia, PA, USA, February 25 - March 4, 2025*, Toby Walsh, Julie Shah, and Zico Kolter (Eds.). AAAI Press, 27529–27537. doi:10.1609/AAAI.V39I26.34965

[38] Prakash Murali, Jonathan M. Baker, Ali Javadi-Abhari, Frederic T. Chong, and Margaret Martonosi. 2019. Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019, Providence, RI, USA, April 13-17, 2019*, Iris Bahar, Maurice Herlihy, Emmett Witchel, and Alvin R. Lebeck (Eds.). ACM, 1015–1029. doi:10.1145/3297858.3304075

[39] Prakash Murali, Norbert Matthias Linke, Margaret Martonosi, Ali Javadi-Abhari, Nhung Hong Nguyen, and Cinthia Huerta Alderete. 2019. Full-stack, real-system quantum computer studies: architectural comparisons and design insights. In *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22-26, 2019*, Srilatha Bobbie Manne, Hillery C. Hunter, and Erik R. Altman (Eds.). ACM, 527–540. doi:10.1145/3307650.3322273

[40] Ramon Serna Oliver, Silviu S. Craciunas, and Wilfried Steiner. 2018. IEEE 802.1Qbv Gate Control List Synthesis Using Array Theory Encoding. In *IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2018, 11-13 April 2018, Porto, Portugal*, Rodolfo Pellizzoni (Ed.). IEEE Computer Society, 13–24. doi:10.1109/RTAS.2018.00008

[41] Sebastian Poeplau and Aurélien Francillon. 2020. Symbolic execution with SymCC: Don't interpret, compile!. In *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, Srdjan Capkun and Franziska Roesner (Eds.). USENIX Association, 181–198. https://www.usenix.org/conference/usenixsecurity20/presentation/poeplau

[42] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*, Jan Vitek, Haibo Lin, and Frank Tip (Eds.). ACM, 335–346. doi:10.1145/2254064.2254104

[43] Carolyn B. Seaman. 1999. Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Trans. Software Eng.* 25, 4 (1999), 557–572. doi:10.1109/32.799955

[44] Roberto Sebastiani and Patrick Trentin. 2015. OptiMathSAT: A Tool for Optimization Modulo Theories. In *Computer Aided Verification - 27th International Conference, CAV 2015, San Francisco, CA, USA, July 18-24, 2015, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 9206)*, Daniel Kroening and Corina S. Pasareanu (Eds.). Springer, 447–454. doi:10.1007/978-3-319-21690-4_27

[45] Sanjit A. Seshia, Dorsa Sadigh, and S. Shankar Sastry. 2022. Toward verified artificial intelligence. *Commun. ACM* 65, 7 (2022), 46–55. doi:10.1145/3503914

[46] Yan Shoshitaishvili, Ruoyu Wang, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. 2015. Firmalice - Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015*. The Internet Society. https://www.ndss-symposium.org/ndss2015/firmalice-automatic-detection-authentication-bypass-vulnerabilities-binary-firmware

[47] Maolin Sun, Yibiao Yang, Ming Wen, Yongcong Wang, Yuming Zhou, and Hai Jin. 2023. Validating SMT Solvers via Skeleton Enumeration Empowered by Historical Bug-Triggering Inputs. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 69–81. doi:10.1109/ICSE48619.2023.00018

[48] Maolin Sun, Yibiao Yang, Jiangchang Wu, and Yuming Zhou. 2025. Validating SMT Rewriters via Rewrite Space Exploration Supported by Generative Equality Saturation. *Proc. ACM Program. Lang.* 9, OOPSLA2 (2025), 1205–1231. doi:10.1145/3763093

[49] Maolin Sun, Yibiao Yang, and Yuming Zhou. 2026. Once4All: Skeleton-Guided SMT Solver Fuzzing with LLM-Synthesized Generators. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2026, Pittsburgh, PA, USA, March 22-26, 2026*, Benjamin C. Lee, Harry Xu, Mark Silberstein, and Bingyao Li (Eds.). ACM, 1316–1332. doi:10.1145/3779212.3790195

[50] Nestan Tsiskaridze, Clark W. Barrett, and Cesare Tinelli. 2024. Generalized Optimization Modulo Theories. In *Automated Reasoning - 12th International Joint Conference, IJCAR 2024, Nancy, France, July 3-6, 2024, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14739)*, Christoph Benzmüller, Marijn J. H. Heule, and Renate A. Schmidt (Eds.). Springer, 458–479. doi:10.1007/978-3-031-63498-7_27

[51] Nestan Tsiskaridze, Andrew Reynolds, Cesare Tinelli, and Clark Barrett. 2025. A Proposal for an OMT Extension to SMT-LIB. In *Proceedings of the 23rd International Workshop on Satisfiability Modulo Theories (SMT 2025) co-located with the 28th International Conference on Theory and Applications of Satisfiability Testing (SAT 2025), Glasgow, UK, August 10–11, 2025*. CEUR-WS.org, 29–44. https://ceur-ws.org/Vol-4008/SMT_paper25.pdf

[52] Dominik Winterer and Zhendong Su. 2024. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 2378–2401. doi:10.1145/3689795

[53] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. On the unusual effectiveness of type-aware operator mutations for testing SMT solvers. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 193:1–193:25. doi:10.1145/3428261

[54] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. Validating SMT solvers via semantic fusion. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 718–730. doi:10.1145/3385412.3385985

[55] Yibiao Yang, Maolin Sun, Jiangchang Wu, Qingyang Li, and Yuming Zhou. 2025. Debugger Toolchain Validation via Cross-Level Debugging. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 280–294. doi:10.1145/3669940.3707271

[56] Peisen Yao, Qingkai Shi, Heqing Huang, and Charles Zhang. 2021. Program analysis via efficient symbolic abstraction. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–32. doi:10.1145/3485495

[57] Samee Zahur and David Evans. 2013. Circuit Structures for Improving Efficiency of Security and Privacy Tools. In *2013 IEEE Symposium on Security and Privacy, SP 2013, Berkeley, CA, USA, May 19-22, 2013*. IEEE Computer Society, 493–507. doi:10.1109/SP.2013.40

[58] Danfeng Zhang and Daniel Kifer. 2017. LightDP: towards automating differential privacy proofs. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, POPL 2017, Paris, France, January 18-20, 2017*, Giuseppe Castagna and Andrew D. Gordon (Eds.). ACM, 888–901. doi:10.1145/3009837.3009884

[59] Junda Zheng and Peisen Yao. 2025. Bounded-Exhaustive Subspace Diversification for SMT Solver Testing. In *Static Analysis - 32nd International Symposium, SAS 2025, Singapore, October 13-14, 2025, Proceedings (Lecture Notes in Computer Science, Vol. 16100)*, Hakjoo Oh and Yulei Sui (Eds.). Springer, 167–189. doi:10.1007/978-3-032-07106-4_8

Received 2026-03-18; accepted 2026-08-06